

Rückgabewerte vs. Effekte

1 Wdh. Definition von Klassen

Mit dem folgenden Code wird eine Klasse Student mit den Eigenschaften

- name vom Typ String
- age vom Typ int

definiert. Außerdem wird ein Konstruktor definiert, dem beide Werte übergeben werden.

```
class Student {  
    String name;  
    int age;  
  
    public Student(String name, int age){  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
var pana = new Student("Pana", 17);
```

2 Methoden mit Rückgabewerten

Wir können in dieser Klasse eine Methode implementieren, die das nächste Alter eines Objekts berechnet.

```
int nextAge(){  
    return age + 1;  
}
```

```
pana.nextAge()
```

Ein Aufruf dieser Methode berechnet das neue Alter. Das Objekt, auf dem die Methode aufgerufen wird, ändert sich dabei aber nicht.

```
pana.age
```

17

Bei der nächsten Methode wird ein neuer Student erzeugt, der ein Jahr älter ist.

```
Student nextStudent(){  
    return new Student(name, age);  
}
```

```
var nextPana = pana.nextStudent();
```

```
nextPana.name;
```

Pana

```
nextPana.age;
```

17

Auch beim Aufruf dieser Methode ändert sich das Objekt, auf dem die Methode aufgerufen wird, nicht.

```
pana.age;
```

17

3 Methoden, die Objekte verändern

Die Methode `getOlder` gibt keinen Wert zurück. Sie ändert aber die Eigenschaft `age` des Objekts, auf dem die Methode aufgerufen wird. Da in der Kopfzeile nur der Rückgabetypp angegeben wird, steht hier `void`.

```
void getOlder(){  
    age = age + 1;  
}
```

```
pana.getOlder()
```

```
pana.age
```

18

4 Methoden, die Objekte verändern und Werte zurückgeben

Es ist auch möglich, dass eine Methode das Objekt, auf dem sie aufgerufen wird, verändert **und** einen Wert zurückgibt.

```
public bool getOlderCheckAllowedToBuyBeer(){  
    age = age + 1;  
    return age >= 16;  
}
```

```
var alex = new Student("Alex", 15);
```

```
alex.getOlderCheckAllowedToBuyBeer()
```

true

```
alex.age
```

16